



S.E.P. TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO de Tuxtepec

Programación Nativa Para Plataforma Móviles

ACTIVIDAD:

“reporte introducción a flutter”

PRESENTA:

LIDIA DUBLAN ZALAZAR

CONTROL:

22350396

CARRERA:

INGENIERIA INFORMÁTICA

ESPECIALIDAD:

DESARROLLO DE SISTEMAS MÓVILES CON
TECNOLOGÍA ACTUALES

SEMESTRE: 8

DOCENTE:

JULIO AGULAR CARMONA

12/06/2026

Definición Formal: Qué es Flutter (el SDK de código abierto de Google) y la naturaleza de **Dart** como su lenguaje de programación nativo, explicando cómo compila directamente a código de máquina (AOT y JIT).

Características Clave: Explicación técnica de su arquitectura basada en widgets (todo es un widget), el motor gráfico **Impeller / Skia** (que rinde a 60/120 fps sin depender de puentes nativos como otras tecnologías) y el Hot Reload.

Ventajas: Desarrollo multiplataforma con un único codebase, rendimiento cercano al nativo, alta capacidad de personalización de UI y una comunidad respaldada por Google.

Desventajas: Tamaño de los archivos binarios (APKs/IPAs más pesados), madurez frente al desarrollo nativo puro en APIs muy específicas de hardware, y la curva de aprendizaje de Dart.

Importancia: Por qué es crucial en la materia de Programación Nativa para Plataformas Móviles, optimizando costos y tiempos de desarrollo en la ingeniería de software actual.

Ejemplos en el Mercado: Casos de éxito reales como Alibaba, Hamilton Musical, Google Ads y Reflectly, demostrando su viabilidad comercial.

Sección 1: Introducción (Introduction)

¿De qué trata?: Es la bienvenida y presentación del curso. Es impartido por colaboradores de DSC VIT (Developer Student Clubs) con el apoyo de Google Developers.

Objetivo: Conocer a los instructores, entender la ruta de aprendizaje y comprender qué tipo de aplicaciones móviles se pueden construir desde cero con el framework.

Desarrollo Técnico: En esta sección de apertura se estableció el mapa de ruta del curso impartido por desarrolladores de DSC VIT. Se analizó el panorama actual del desarrollo móvil, comprendiendo cómo Flutter mitiga la brecha histórica entre el

desarrollo nativo de Android (Java/Kotlin) y Apple (Swift), permitiendo compilar para ambas plataformas desde un único código base sin sacrificar el rendimiento gráfico.

Sección 2: Configuración del Entorno (Environment Setup)

¿De qué trata?: Se enfoca en preparar tu computadora para poder programar.

Objetivo: Aprender el proceso de instalación del SDK de Flutter, la configuración de las variables de entorno del sistema y la preparación del editor de código (como VS Code o Android Studio) junto con los emuladores o dispositivos físicos para pruebas.

Desarrollo Técnico: Se realizó el proceso práctico de preparación del entorno de desarrollo. Esto incluyó la descarga y extracción del SDK de Flutter, la configuración de las variables de entorno en el sistema operativo para habilitar los comandos en la terminal (flutter doctor), y la instalación de extensiones clave en el editor de código (Visual Studio Code / Android Studio). Asimismo, se configuraron las herramientas de virtualización para desplegar emuladores locales y se habilitó el modo de depuración por USB en un dispositivo físico para pruebas de rendimiento en tiempo real.

Sección 3: Primeros Pasos (Getting Started)

¿De qué trata?: Tu primer contacto real con el código y la estructura de un proyecto en Flutter.

Objetivo: Crear una interfaz básica desde cero.

Entender cómo organizar el código creando clases independientes (buenas prácticas de arquitectura).

Aprender a añadir las primeras líneas de lógica para que el diseño no sea estático.

Desarrollo Técnico: Se analizó la arquitectura inicial de un proyecto de Flutter, identificando el archivo main.dart como el punto de entrada de la aplicación. Se aprendió a estructurar el código de forma limpia implementando la programación orientada a objetos (POO), abstrayendo las interfaces en clases independientes para asegurar la modularidad, escalabilidad y fácil mantenimiento del software.

Sección 4: Widgets Comunes y de Uso Frecuente (Commonly used widgets)

¿De qué trata?: Esta es una de las secciones más importantes, ya que en Flutter "todo es un widget". Aquí se analiza el catálogo de componentes visuales esenciales.

Objetivo: Dominar el diseño de interfaces utilizando:

AppBar, Icon, Center: Para crear barras superiores, navegación y centrar elementos.

Colors, Container, Gradients: Para dar estilos, fondos, cajas contenedoras y degradados.

Gesture Detectors y SnackBar: Para detectar toques/gestos en la pantalla y mostrar mensajes flotantes informativos.

Text, Row, Column, SingleChildScrollView: Para estructurar texto y alinear elementos de forma horizontal (Row) o vertical (Column), añadiendo scroll para evitar que la pantalla se desborde.

Flexible vs Expanded: Control del espacio y cómo se expanden los componentes en pantallas de diferentes tamaños.

ListTile, ListViews y Padding: Para crear listas dinámicas con iconos, títulos y márgenes limpios.

Desarrollo Técnico: Se dominó el paradigma de que en Flutter "todo es un widget". Se implementaron componentes estructurales como Scaffold, AppBar, Center e Icon. Para la distribución espacial y diseño adaptivo, se utilizaron contenedores (Container), filas (Row) y columnas (Column). Con el fin de evitar desbordamientos de memoria y de interfaz (*overflow*), se aplicaron widgets de scroll como SingleChildScrollView y optimizaciones de espacio mediante Flexible y Expanded. Finalmente, se construyeron interfaces basadas en colecciones utilizando ListView y ListTile con márgenes controlados por Padding.

Sección 5: Widgets con Estado y sin Estado (Stateful and Stateless Widgets)

¿De qué trata?: Explica el núcleo de la arquitectura del manejo de estados en Flutter.

Objetivo: Comprender la diferencia teórica y práctica entre un **StatelessWidget** (una interfaz estática que no cambia sus datos una vez construida) y un **StatefulWidget** (una interfaz dinámica que puede redibujarse en tiempo real cuando el usuario interactúa, por ejemplo, al presionar un botón o cambiar un contador).

Desarrollo Técnico: Se profundizó en el ciclo de vida de los componentes. Los StatelessWidget se aplicaron en interfaces puramente informativas o estáticas que no sufren mutaciones de datos en tiempo de ejecución. Por otro lado, se implementó el uso de StatefulWidget para pantallas dinámicas. Se comprendió el funcionamiento del método `setState()`, el cual notifica al framework que el estado interno ha cambiado, forzando un redibujado eficiente del subárbol de widgets para reflejar las nuevas interacciones del usuario en tiempo real.

Sección 6: Trabajando con Formularios (Working with Forms)

¿De qué trata?: Captura e interacción de datos introducidos por el usuario.

Objetivo: Implementar elementos de formulario (`TextFormField`), aplicar lógica de **validación** (para asegurar que el usuario no deje campos vacíos o introduzca correos inválidos) y gestionar el **envío seguro** de esos datos internamente en la app.

Desarrollo Técnico: Se desarrollaron módulos de captura de datos utilizando el widget `Form` y campos de texto estructurados (`TextFormField`). Se implementó lógica de validación mediante llaves globales (`GlobalKey<FormState>`) para verificar la integridad de las entradas (campos vacíos, formatos de correo electrónico, etc.) antes de permitir el procesamiento o envío seguro de la información.

Sección 7: Navegación entre Pantallas (Navigation through screens)

¿De qué trata?: Gestión de rutas y flujos de usuario dentro de la aplicación.

Objetivo: Dominar el sistema de navegación basado en una pila (Stack), utilizando funciones como:

`push()` y `pop()`: Para ir a una nueva pantalla y regresar a la anterior.

`pushReplacementNamed()` y `popAndPushNamed()`: Para cambiar de pantalla destruyendo la anterior (ideal para pantallas de Login o Splash Screens).

`popUntil()` y `pushNamedAndRemoveUntil()`: Para limpiar el historial de navegación hasta llegar a una pantalla específica, protegiendo el flujo de datos.

Desarrollo Técnico: Se administró el flujo de las interfaces mediante el enrutamiento de pantallas basado en una estructura de pila (Stack). Se dominó el uso de `Navigator.push()` para apilar una nueva pantalla y `Navigator.pop()` para retornar a la anterior. Para flujos de seguridad y control de sesión (como pantallas de carga o inicios de sesión), se implementaron métodos avanzados como `pushReplacementNamed()`, `popUntil()` y `pushNamedAndRemoveUntil()`, los cuales limpian o reemplazan el historial de navegación para evitar que el usuario regrese a pantallas críticas de manera incorrecta.

Sección 8: Procesamiento de Datos JSON (JSON Parsing)

¿De qué trata?: Preparación para que la aplicación consuma información de internet.

Objetivo: Aprender a interpretar y deserializar datos en formato JSON (el estándar de la web) en objetos de Dart. Cubre desde estructuras JSON simples, pasando por JSON que contienen arreglos (listas), hasta estructuras JSON anidadas y complejas.

Desarrollo Técnico: Se abordó la comunicación de datos mediante la manipulación del formato estándar JSON. Se aprendió a deserializar cadenas de texto planas provenientes de servicios web en mapas y objetos fuertemente tipados en Dart. Se desarrollaron algoritmos capaces de procesar desde estructuras JSON simples, hasta arreglos de datos complejos y estructuras anidadas de múltiples niveles.

Sección 9: Proyecto Práctico: Aplicación de Terremotos (Earthquake App)

¿De qué trata?: El proyecto final e integrador del curso.

Objetivo: Construir una aplicación funcional desde cero que realice peticiones HTTP para obtener datos reales sobre sismos desde una API del gobierno de EE. UU. Aquí se pone en práctica el widget **FutureBuilder**, el cual se encarga de gestionar de forma asíncrona la carga de datos de internet, mostrando un indicador de carga mientras recibe la información y pintando la interfaz final con la lista de terremotos una vez completada la petición.

Desarrollo Técnico: Como proyecto integrador, se consolidaron los conocimientos mediante el desarrollo de una aplicación conectada a la API pública de sismos del gobierno de los Estados Unidos. Se programó un cliente HTTP para realizar peticiones asíncronas de tipo GET. La renderización de la interfaz se gestionó mediante el widget **FutureBuilder**, el cual evalúa el estado de la conexión (`ConnectionState.waiting`) para mostrar un indicador visual de carga (como un `CircularProgressIndicator`) y, una vez que los datos JSON son parseados con éxito, dibuja dinámicamente un `ListView` con la información de los sismos en tiempo real.

Conclusion

El desarrollo de este curso introductorio a Flutter permitió comprender de manera práctica los retos y las soluciones modernas en la ingeniería de software enfocada al entorno móvil. Como estudiante de la carrera de Ingeniería Informática, resulta de vital importancia dominar herramientas que optimicen los ciclos de desarrollo. Flutter demuestra ser una tecnología disruptiva; la capacidad de compilar directamente a código de máquina (AOT) mediante Dart garantiza que el rendimiento visual provisto por su motor gráfico no sufra los retardos comunes de otras tecnologías multiplataforma.